

FanfareHub – Tutoriel d’architecture (MVC/DAO)

Projet TP9

12 mai 2026

1 Objectif

Ce document sert de fiche de compréhension du projet Java Web (Tomcat + JSP + Servlets + DAO) pour expliquer le code à l’oral.

2 Architecture globale

Vue (JSP)

Pages d’affichage et formulaires :
`connexion.jsp`, `inscription.jsp`, `menu.jsp`, `groupesPupitres.jsp`, `administration.jsp`, `evenements.jsp`,
`evenementParticipation.jsp`.

Contrôleurs (Servlets, package `metier`)

- `AuthServicelet` : connexion, inscription, logout
- `ChoixServlet` : groupes/pupitres
- `AdminServlet` : CRUD fanfarons (accès admin)
- `EvenementServlet` : événements + participation
- `ProfilServlet` : point d’entrée profil (minimal)

Modèle (package `dao` + **POJO** package `metier`)

- DAO : `DbConnectionManager`, `FanfaronDAO`, `PupitreDAO`, `GroupeDAO`, `EvenementDAO`
- POJO : `Fanfaron`, `Pupitre`, `Groupe`, `Evenement`, `InscriptionEvenement`

3 Flux de requêtes importants

Connexion

`connexion.jsp` → `AuthServicelet.doPost(action=connexion)` → `FanfaronDAO.verifIdentif(...)`
→ session (`login`, `role`) → `menu.jsp`.

Groupes/Pupitres (Q2)

`menu.jsp` → `ChoixServlet.doGet` (chargement options + choix actuels) → `groupesPupitres.jsp`.
Submit formulaire → `ChoixServlet.doPost` → `FanfaronDAO.saveChoix(...)` (transaction).

Administration fanfarons

Lien visible seulement si `role=admin`.
`AdminServlet` vérifie côté serveur (403 sinon), puis exécute `add/update/delete` via `FanfaronDAO`.

Événements (Q4/Q5)

`EvenementServlet.doGet(action=list)` charge la liste.

CRUD événement autorisé seulement à la commission prestation via `EvenementDAO.isCommissionPrestationMem`

Participation fanfaron : `action=saveParticipation` → `EvenementDAO.replaceParticipation(...)`.

4 Mapping TP9 vers code

- Q2 (groupes/pupitres) : `ChoixServlet` + `groupesPupitres.jsp` + `FanfaronDAO.saveChoix`
- Q4 (CRUD événements) : `EvenementServlet` + `EvenementDAO.create/update/delete` + `evenements.jsp`
- Q5 (inscription événement) : `evenementParticipation.jsp` + `EvenementServlet.saveParticipation` + `EvenementDAO.findInscriptionsByEvenement`

5 POJO et DAO : comment ça marche

POJO

Un POJO est un objet Java simple (attributs + getters/setters/constructeur), sans logique SQL.

Exemple : `Fanfaron`, `Evenement`.

DAO

Un DAO contient les requêtes SQL. Il lit/écrit la base et transforme les lignes SQL en POJO.

Exemple : `FanfaronDAO.findByNomFanfaron()` retourne un objet `Fanfaron` construit depuis `ResultSet`.

Chaîne complète

Servlet (contrôleur) reçoit la requête → appelle DAO → DAO retourne POJO/listes POJO → Servlet met ça dans `request.setAttribute(...)` → JSP affiche.

6 Points d'attention pour l'oral

- Toujours distinguer **qui fait quoi** : JSP affiche, Servlet orchestre, DAO accède SQL.
- Sécurité : cacher un lien ne suffit pas ; la vérification est faite aussi côté servlet.
- Transactions : utiles pour garder une base cohérente (ex : `saveChoix`).
- Encodage UTF-8 : important pour les accents saisis dans les formulaires.